# UNIVERSIDAD DE LA HABANA

## Facultad de Matemática y Computación

### Asignatura: Sistemas Operativos

# THE MATCOM MINIX

Informe Técnico


**Integrantes**

**Nombre:** Olivia Ortiz Arboláez **Grupo:** 211
**Nombre:** Anthony Cruz García **Grupo:** 211


**Fecha de entrega:** 25 de abril de 2026

# 1. Introducción

El presente trabajo de la asignatura tiene el propósito de profundizar en el aprendizaje del funcionamiento de los sistemas operativos a través de la interacción directa con MINIX, un sistema operativo educativo diseñado por Andrew S. Tanenbaum.
El proyecto se estructura en varias actividades concretas que abarcan desde la instalación y configuración del entorno de trabajo, la personalización del mensaje de bienvenida, la depuración de un bug en la capa de compatibilidad de pthread, la implementación de un comando de línea de comandos (tree) y la introducción de una política de penalización para procesos que consumen excesivamente CPU.
Este informe técnico documenta detalladamente cada una de estas actividades, describiendo el proceso seguido, los resultados obtenidos y las dificultades encontradas, con el objetivo de demostrar la comprensión de los conceptos fundamentales de los sistemas operativos y su aplicación práctica.

# 2. Desarrollo y resultados por componente

Toda esta sección contiene el proceso completo de trabajo: desde el análisis inicial y las decisiones de diseño, pasando por los cambios concretos realizados en el código o la configuración del sistema, hasta la verificación experimental de su correcto funcionamiento.

## 2.1. Instalación y configuración de VirtualBox y MINIX

Para la instalación de Oracle VirtualBox se consideraron los requisitos mínimos recomendados. Dado el carácter educativo del proyecto y las capacidades de las computadoras de los integrantes del equipo, no resultaba conveniente asignar recursos excesivos a la máquina virtual. En consecuencia, se adoptó una configuración base que ofreciera un desempeño estable sin afectar el funcionamiento del sistema anfitrión.

Como sistema operativo anfitrión se utilizó Ubuntu 24.04 LTS, siguiendo la documentación oficial de VirtualBox durante todo el proceso de instalación. Posteriormente, se descargó la imagen ISO de MINIX 3.3.0 desde su sitio oficial y se creó una nueva máquina virtual con las especificaciones que se detallan a continuación:

- **VirtualBox versión:** 7.0.16
- **MINIX versión:** 3.3.0
- **RAM asignada:** 1024 MB

- **CPU:** 1 núcleo
- **Disco duro:** 2 GB (reserva dinámica)
- **Red:** [NAT]

### Inconvenientes y soluciones

Durante la instalación de MINIX en VirtualBox, se presentaron algunos inconvenientes como el desconocimiento de que VirtualBox seguía iniciando desde el CD virtual (el archivo ISO de instalación) en lugar de hacerlo desde el disco duro virtual donde ya se había instalado el sistema MINIX. Esto se solucionó entrando a la configuración de la máquina virtual, dirigiéndose a la sección de almacenamiento, seleccionando el ISO y eliminando el disco de la unidad virtual, para que el sistema iniciara correctamente.

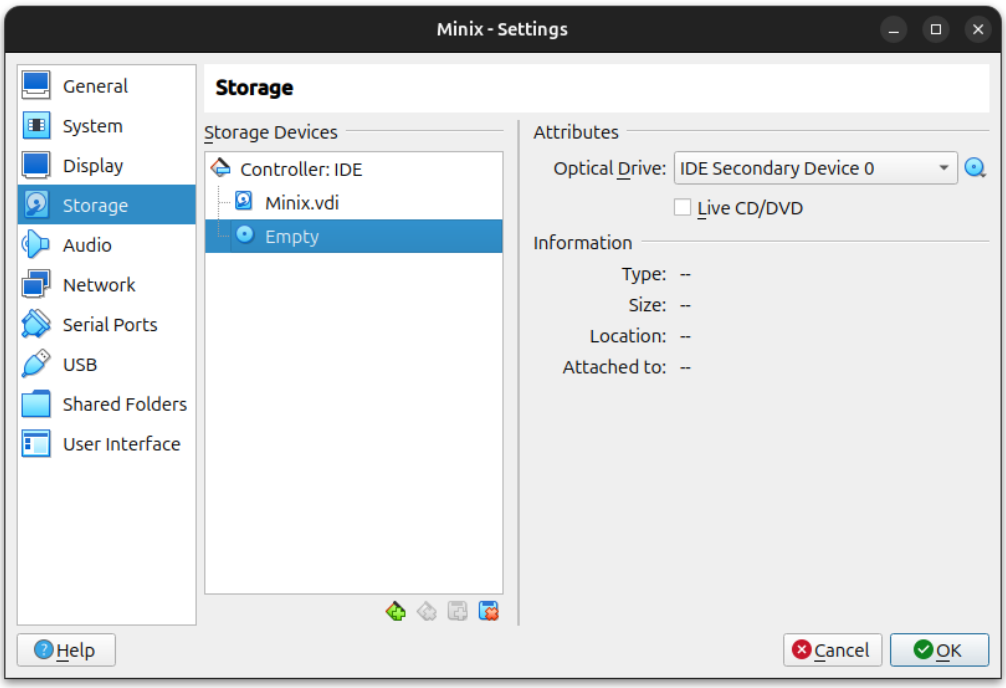



Figura 1: Eliminado el ISO de instalación para iniciar desde el disco virtual

## 2.2. Personalización del mensaje de bienvenida

Aquí se muestra el proceso de modificación del archivo **motd** (Message of the Day) para que, al iniciar sesión en MINIX, se muestre un mensaje personalizado que incluya una referencia a MATCOM.

### Archivo modificado

`/etc/motd`

### Permisos

Para mostrar los permisos del archivo, se empleó el comando `ls -l /etc/motd`, obteniendo la siguiente salida:

```
-rwxr-xr-x 1 root operator 171
```

La interpretación de esta salida es la siguiente:

- **Permisos (`-rwxr-xr-x`):** el primer carácter (`-`) confirma que se trata de un archivo común. Los siguientes tres caracteres indican que el propietario dispone de permisos de lectura, escritura y ejecución (`rwx`); los tres siguientes muestran que el grupo solo tiene lectura y ejecución (`r-x`); y los últimos tres señalan que el resto de los usuarios poseen los mismos permisos que el grupo, sin permiso de escritura.
- **Número de enlaces (`1`):** representa la cantidad de hard links que apuntan al inodo de este archivo específico.
- **Propietario (`root`):** indica que el usuario con privilegios de administración es el dueño del archivo.
- **Grupo (`operator`):** identifica el grupo al cual está asignado el archivo, cuyos miembros están sujetos a los permisos de grupo definidos anteriormente.
- **Tamaño (`171`):** especifica que el archivo ocupa exactamente 171 bytes de almacenamiento en el disco.

### Editor utilizado

Se utilizó el editor de texto nativo de Minix, `mined` con el comando:

```
mined /etc/motd
```

### Contenido agregado

```
¡¡¡ Le damos la Bienvenida a MINIX 3 !!!

-------------- FACULTAD DE MATEMATICA Y COMPUTACION --------------

Universidad de La Habana -- Proyecto de Sistemas Operativos 2026
```

### Verificación

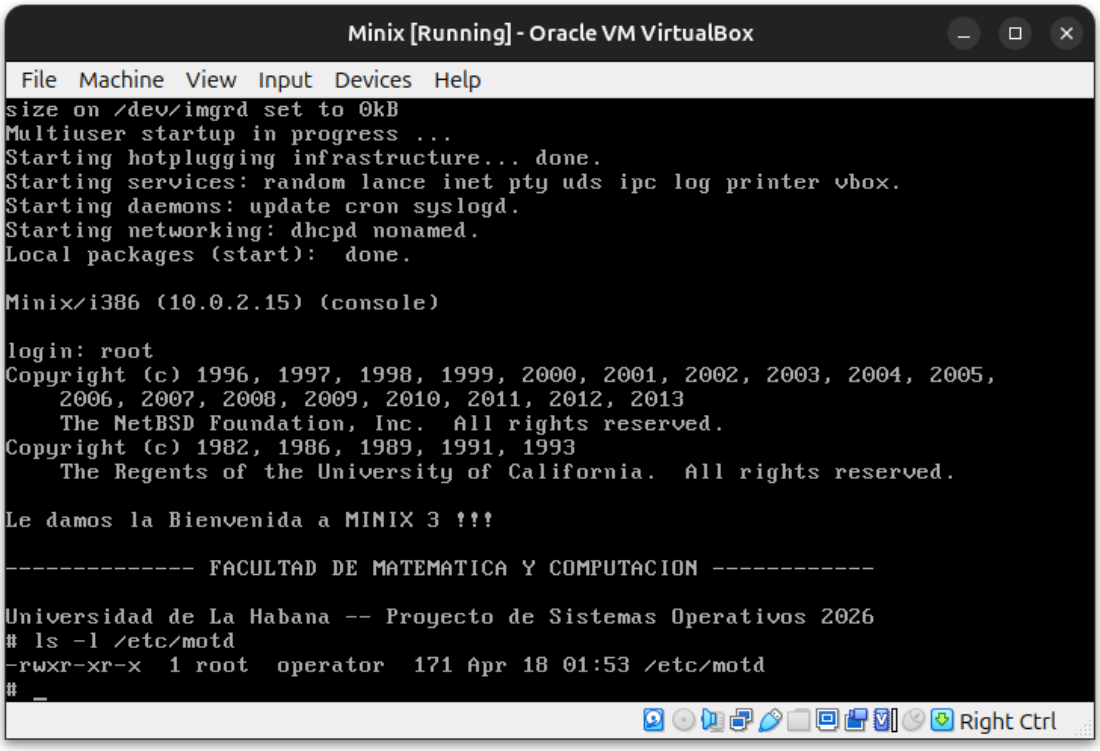



Figura 2: Verificación del mensaje de bienvenida personalizado en `/etc/motd`

## 2.3. Depuración de un bug en pthread

Se nos asignó la tarea de depurar un bug en la capa de compatibilidad de pthread, específicamente el archivo `libmthread/pthread_compat.c`. Este archivo básicamente actúa como un puente entre la API (Application Programming Interface) de pthread y la implementación nativa de hilos en MINIX. Es decir, convierte o traduce los pthreads en mthreads de MINIX, permitiendo que se pueda utilizar el API estándar de POSSIX con la implementación que posee MINIX.

Figura 3: Capa de compatibilidad de pthread

### Síntomas

Para ejecutar el programa de prueba, se utilizó el compilador `clang` con el siguiente comando:

```
clang test_bug.c -o test_bug -lmthread
```

Esto generó el archivo ejecutable `test_bug`, que al ser ejecutado, se observó que el programa se bloqueaba. Añadimos un mensaje al `test_bug.c` para verificar que se llamaba al `pthread_mutex_trylock()`, y efectivamente, ocurría esto.

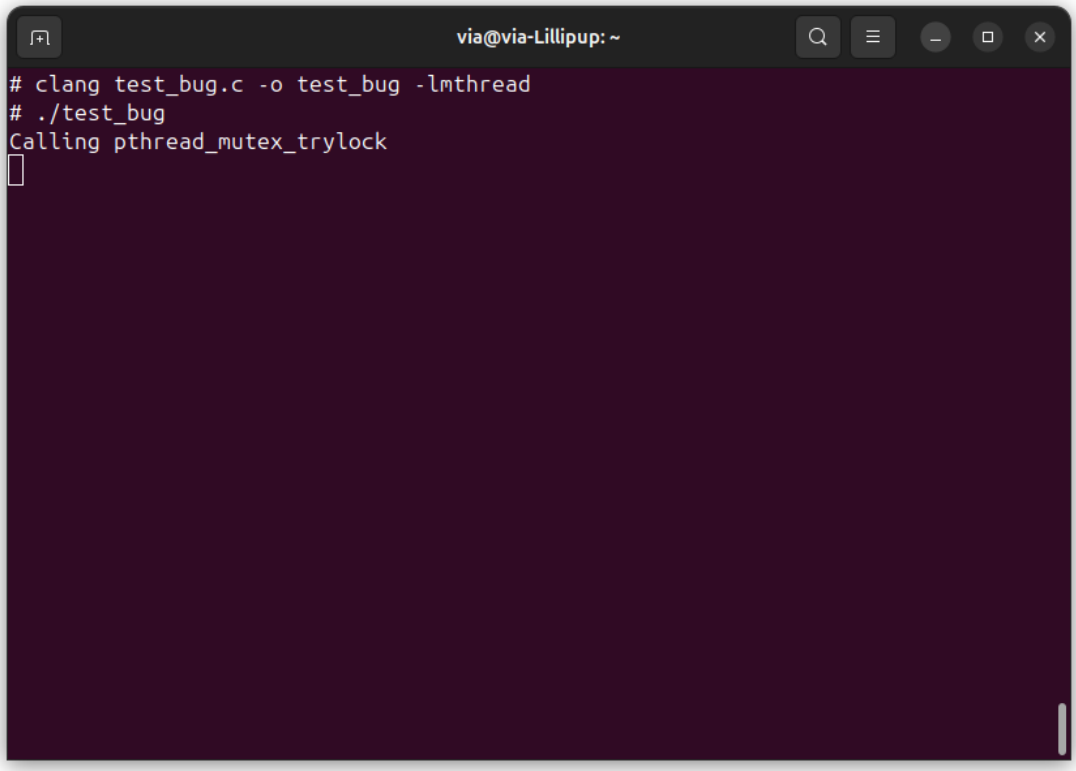



Figura 4: Bloqueo del programa en `pthread_mutex_trylock`

**Análisis**

- Al revisar el código de `pthread_mutex_trylock()` en `pthread_compat.c`, notamos una recursión sin fin.
- En lugar de llamar a la función nativa del sistema [ `mthread_mutex_trylock()` que realiza el trabajo real ], la función `pthread_mutex_trylock()` se invoca a sí misma.
- Debido a esta recursión, el hilo de ejecución queda atrapado y la función nunca llega a una instrucción de retorno (`return`).
- Al no existir una condición de parada, la recursión consume toda la memoria de la pila (*stack*) asignada al proceso.
- Esto eventualmente provoca un error de Out of memory (Código 12).
- El error rompe el propósito de la función, que es traducir las llamadas de pthreads a mthreads.

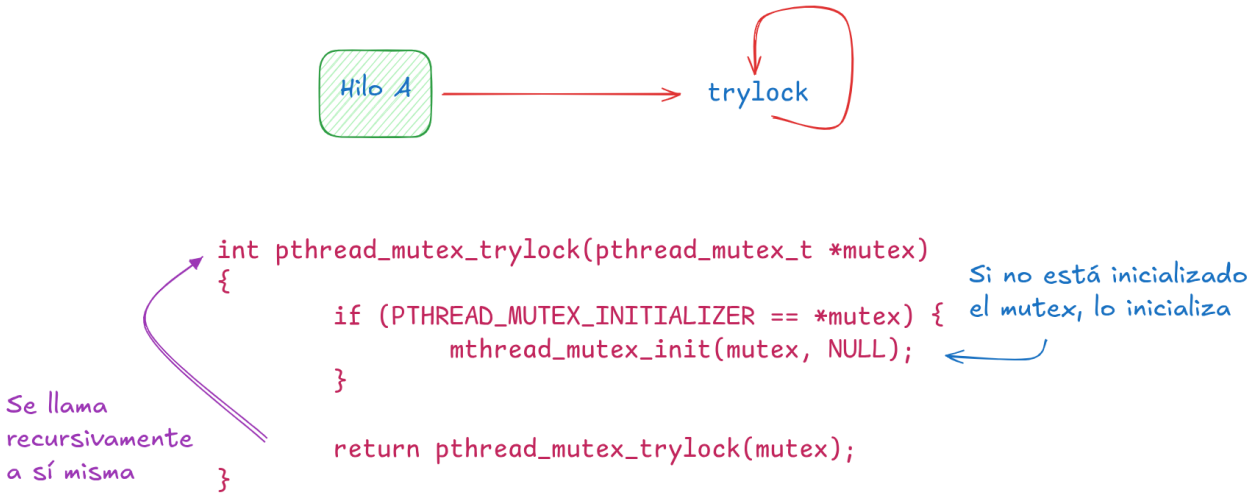



Figura 5: Representación del bug en la capa de compatibilidad de pthread

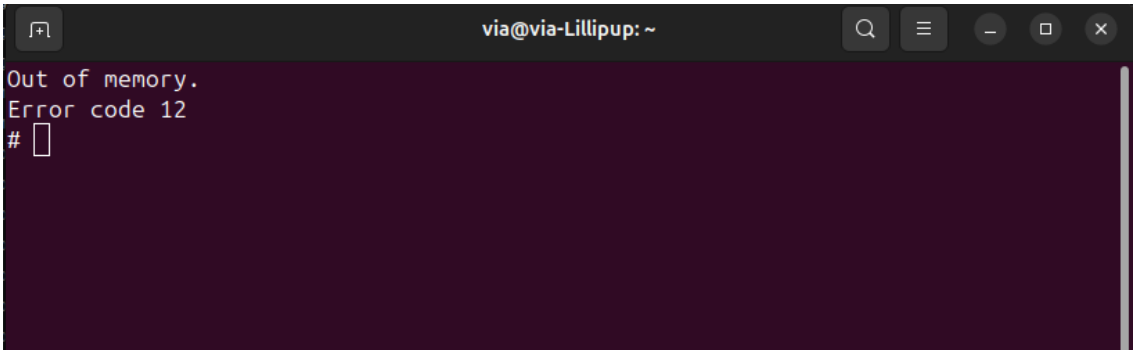



Figura 6: Error 12: Out of memory durante la ejecución del test

### Corrección

La solución técnica mínima para corregir este bug que utilizamos consiste en cambiar la última línea para que llame a la función nativa.
Esta función (`mthread_mutex_trylock`) hace, en esencia, lo siguiente:

- Intenta tomar el mutex de inmediato. Si está libre, lo adquiere y retorna `0` (OK).
- No bloquea al hilo. Si el mutex está ocupado, retorna al instante un código de error.
- Detecta interbloqueo. Si un hilo intenta hacer `trylock` sobre un mutex que ya posee, retorna `EDEADLK` (35: Resource deadlock avoided).
- Si intenta hacer `trylock` sobre un mutex que otro hilo posee, retorna `EBUSY` (16: Device or resource busy).
- En MINIX, `pthread_*` actúa como envoltorio. Por eso, `pthread_mutex_trylock` debe delegar en `mthread_mutex_trylock` y no llamarse recursivamente.

Es diferente al lock tradicional, que bloquea al hilo hasta que el mutex esté disponible. En cambio, `trylock` es no bloqueante y permite al hilo decidir qué hacer si el mutex no está disponible.

```
/*===========================================================================*
 *                                pthread_mutex_trylock                      *
 *===========================================================================*/
int pthread_mutex_trylock(pthread_mutex_t *mutex)
{
        if (PTHREAD_MUTEX_INITIALIZER == *mutex) {
                mthread_mutex_init(mutex, NULL);
        }

        return mthread_mutex_trylock(mutex);
}
```

Figura 7: Código corregido de `trylock`

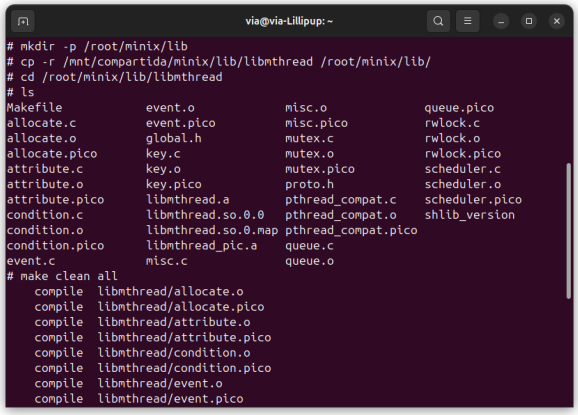
### Verificación

Después de corregir el código, tuvimos que recompilar el código fuente de `lib` para verificar que el bug se había solucionado.
Para esto, fue necesario copiar la carpeta `lib` a un directorio dentro del sistema Minix como tal, puesto que al emplear Carpeta Compartida, nos encontramos con la limitación de que no soporta enlaces simbólicos (ln), este problema nos impedía utilizar el `make install`.

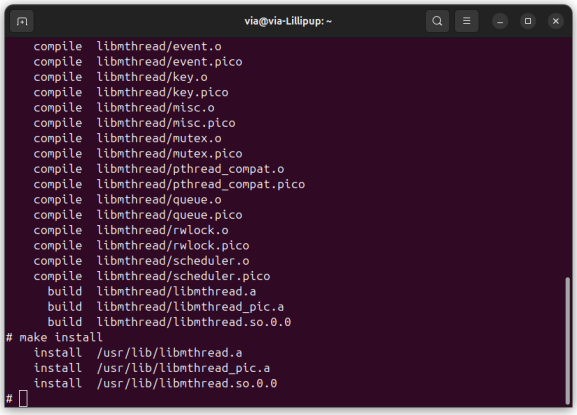
Esto sucede porque las Carpetas Compartidas no fueron diseñadas para ser usadas como un sistema de archivos nativo.
Se utilizaron los comandos:

- `make clean all`[1]
- `make install` [2]

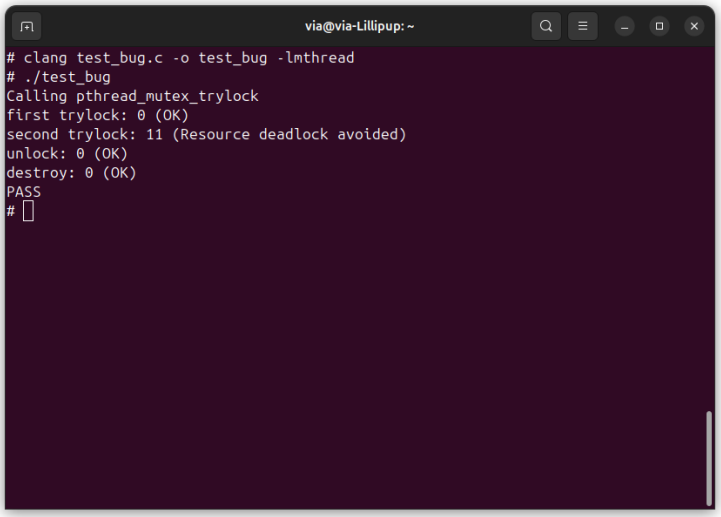
Esto se hizo dentro del directorio `minix/lib`, lo que generó la nueva versión de la biblioteca con la corrección aplicada. Luego, comprobamos compilando el `test_bug.c`: el programa se ejecutaba sin bloquearse, y los códigos de retorno eran los esperados.




Ejecución de `make clean all`




Ejecución de `make install`




Verificación final: el test se ejecuta correctamente

[1] `make clean all` elimina los archivos generados previamente y recompila el proyecto desde cero

[2] `make install` copia la biblioteca resultante en su ubicación de instalación para usar la versión corregida en todo el sistema.

```
first trylock: 0 (OK)
second trylock: 11 (Resource deadlock avoided)
unlock: 0 (OK)
destroy: 0 (OK)
PASS
```

# 3. Referencias consultadas


- Tanenbaum, A. S., & Bos, H. (2015). *Modern Operating Systems* (4th ed.). Pearson.
- Documentación oficial de MINIX. `https://wiki.minix3.org/`
- Repositorio del proyecto. `https://github.com/chriss8g/minix`


# 4. Declaración de uso de IA

## Partes del proyecto donde se utilizó IA

- Se utilizó DeepSeek como apoyo para consultar el procedimiento de acceso por SSH a la máquina virtual de MINIX desde el sistema anfitrión.
- Se utilizó Copilot para explorar posibles causas del problema de `git clone` en MINIX; no se obtuvo una solución definitiva por esta vía.
- Se empleó DeepSeek para orientar la configuración de la Carpeta Compartida entre VirtualBox y MINIX.
- Se utilizó IA de apoyo para mejorar la redacción y claridad de algunos apartados técnicos del informe.

## Naturaleza del uso

- Consulta técnica y orientación de comandos/procedimientos (uso de SSH y Carpeta Compartida).
- Apoyo en diagnóstico preliminar de problemas (caso de `git clone` en MINIX).
- Mejora de redacción técnica y organización del texto.
- La implementación, pruebas y validación final de los resultados fueron realizadas por los integrantes del equipo.

# 5. Anexos

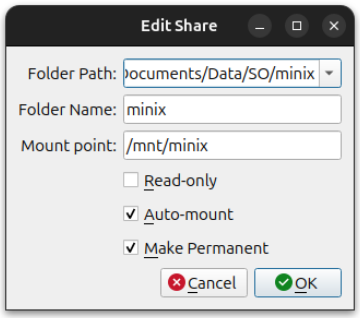
## Enlace al repositorio

`https://github.com/ViA8604/minix.git`

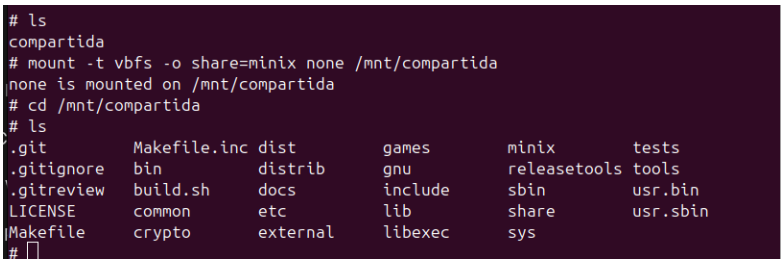
## Capturas de pantalla

| Dónde estás | Qué haces | Para qué sirve |
|---|---|---|
| **Dentro de Minix** (Terminal fea) | Escribes: `pkgin install openssh` | Instalas el **Portero Automático** dentro de Minix. |
| **Dentro de VirtualBox** (Configuración) | Pones "NAT" y "Reenvío de Puertos" | **Conectas el cable del timbre** para que cuando toques en Ubuntu, suene en Minix. |
| **Dentro de Ubuntu** (Terminal bonita) | Escribes: `ssh root@127.0.0.1 -p 2222` | **Tocas el timbre**. El portero te abre la puerta y ves la pantalla de Minix dentro de Ubuntu. |

Consulta sobre acceso SSH




Configuración de Carpeta Compartida




Carpeta Compartida montada en MINIX